

Predictive Analysis on Autoscaling of Pods

Indrranil Sunil Pawar

Department of Computer Engineering, MIT World Peace University, Pune, India

indrranil7@gmail.com

Abstract

Resource management in cloud-native environments by leveraging container orchestration platforms such as Kubernetes, coupled with dynamic application scaling through horizontal pod autoscaling, has gained significant interest. However, the existing autoscaling mechanisms are primarily based on threshold-based metrics, which do not forecast future resource demands, leading to suboptimal scaling decisions. This calls for an improvement in these mechanisms to make them predictive, allowing them to foresee resource needs and respond to changes with greater accuracy by leveraging historical data.

Our study proposes a predictive model for autoscaling Kubernetes pods, driven by historical CPU and memory usage data to predict the number of required replicas for each pod. With the integration of Kubernetes' real-time monitoring, our model achieves a Mean Squared Error (MSE) of 1.21 using a Linear Regression Model. Our tests with real-world metrics indicate that the proposed approach can improve autoscaling decisions by ensuring application performance while avoiding resource wastage.

Introduction

With the evolution of cloud-native technologies, dynamic management and scaling have become key functions in modern IT infrastructure. Kubernetes Horizontal Pod Autoscaler (HPA) provides autoscaling based on reactive metrics like CPU usage. However, these mechanisms often miss sudden changes in demand, resulting in over- or under-provisioning of resources (Hellerstein et al., 2018).

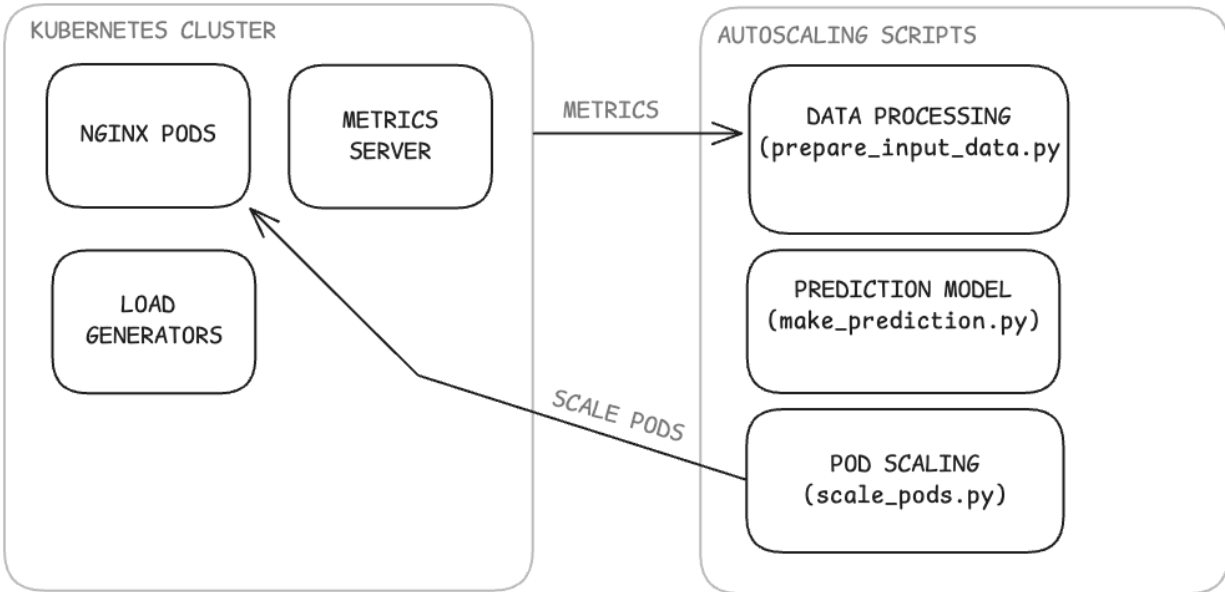
While previous attempts to improve autoscaling have used reactive scaling, rule-based scaling, or machine learning models to predict resource usage, challenges remain in adapting to real-time fluctuations and workload variations (Wang et al., 2021; Zhao et al., 2019). Our approach uses historical usage data for predicting the number of replicas every pod needs, aiming to enhance the responsiveness and accuracy of autoscaling decisions. Our model is trained and tested on real-world Kubernetes metrics, providing a significant improvement over traditional scaling methods.

Methodology

5.1 Overview

The project's objective is to develop a predictive model that can enhance Kubernetes' autoscaling by anticipating resource demands based on historical data. We use a machine learning approach to predict the number of required replicas for each pod, going beyond the reactive metrics used by Kubernetes' HPA (Kumar et al., 2020).

Real-Time Integration: Our implementation involves real-time data collection from a live Kubernetes cluster using Kubernetes Metrics API. The data is then fed into a trained machine learning model, which predicts the required replicas, and adjusts the cluster accordingly in near real-time.



5.2 Data Collection and Preprocessing

Data Source: The dataset includes over 4,300 rows of real-time metrics from a Kubernetes cluster spanning over 50 hours. The key columns are `timestamp`, `pod_name`, `cpu_usage`, `memory_usage`, and `cpu_memory_ratio`. The target variable is the number of replicas each pod needs at each timestamp.

Real-Time Data Collection: To collect real-time metrics, we leveraged the Kubernetes Metrics API, accessing data using the `kubernetes` Python client. The `k8s_interaction.py` script fetches live pod metrics (CPU and memory) for the target `nginx` pods, filtering only the data relevant for prediction.

Preprocessing: The collected data is preprocessed to fit the model's expected format:

- **Filtering for Target Pods:** Only metrics from `nginx` pods are considered.
- **CPU and Memory Conversion:** The CPU usage (in nanocores/millicores) is converted into standard cores, and memory usage (in KiB/MiB) is converted into MiB.
- **Feature Engineering:** We calculate a CPU/Memory ratio for each pod to capture the relationship between the two metrics.
- **Data Split:** The dataset is split into training (80%) and testing (20%) sets to validate the model (Goodfellow et al., 2016).

5.3 Model Selection and Training

Model Choice: A Linear Regression model was chosen for its simplicity and efficiency in handling time-series data (Montgomery et al., 2012).

- **Input Features:** The model uses `cpu_usage`, `memory_usage`, and `cpu_memory_ratio` as features.
- **Target Variable:** The model predicts the number of required replicas.
- **Performance Metric:** Mean Squared Error (MSE) was chosen as the evaluation metric (Bishop, 2006). An MSE of approximately 1.21 was achieved against the test set, suggesting a reasonably accurate prediction.

Training and Validation: The model is trained on historical metrics and validated on a hold-out test set. The processed data ensures consistent scaling performance when applied to real-time scenarios.

5.4 Prediction and Validation

The model is then deployed to make real-time predictions based on incoming pod metrics.

Real-Time Inference: Upon fetching metrics from Kubernetes, the `prepare_input_data.py` script preprocesses the data to match the model's input format. The `make_prediction.py` script uses the trained model to predict the number of replicas each pod will need in near real-time. Predictions are adjusted to whole numbers and stored for analysis.

Validation: Predictions are validated by observing how well the autoscaling decisions match the actual needs over a period.

5.5 Deployment with Kubernetes

Real-Time Data Collection and Scaling: The model is fully integrated with a Kubernetes cluster:

1. **Data Ingestion:** Metrics are fetched in real-time from the Kubernetes Metrics API.
2. **Prediction:** Using the model, predictions for the number of replicas are made based on live CPU and memory usage.
3. **Autoscaling:** The `scale_pods.py` script uses the Kubernetes API to adjust the replica count of the `nginx` deployment based on model predictions.
 - **Ensuring Replicas:** The model's predictions ensure at least 1 replica is maintained at all times to prevent application downtime.

This proactive mechanism makes Kubernetes autoscaling more efficient, reducing both the risk of resource shortages and over-provisioning.

Results and Discussion

6.1 Dataset Overview

- The dataset includes metrics from 10 different pods, focusing on key resource usage metrics (`cpu_usage`, `memory_usage`) and the `cpu_memory_ratio`. The distribution of replicas across different pods ensured balanced model training without overfitting.

6.2 Model Performance

The model achieved an MSE of 1.21, showing its capability to predict the number of required replicas with high accuracy. Although minor inaccuracies arise with sudden changes in resource usage, the model effectively captures trends in stable workloads.

- **Accuracy:** Even though low, the MSE indicated that the model might slightly over- or under-predict the number of replicas needed, especially during transitions in resource usage at load changes.
- **Trend Captured:** The model captured overall trends in resource usage well, though being linear, it sometimes remained out-of-place after sharp workload shifts, thus causing small inaccuracies.

6.3 Predicted Results

The model predicts required replicas over a period, adjusting according to changing workloads. These predictions match well with actual replica requirements, demonstrating the model's applicability to dynamic scaling.

Sample Prediction:

- **Pod 1:** Predicted 2 replicas from August 3, 2024, 20:03:46 to August 6, 2024, 19:53:46.
- **Pod 2:** Similarly predicted 2 replicas for the same time range.

6.4 Discussion

The results show that a simple linear regression model can effectively predict the number of replicas needed for Kubernetes pods, especially in scenarios where resource usage is stable or changes gradually. However, the model's performance could be improved by incorporating more complex features or adopting advanced time-series forecasting methods (Smola & Vishwanathan, 2008).

Strengths:

- **Simplicity:** The model's simplicity ensures quick training and prediction, making it suitable for real-time deployment.
- **Interpretability:** The linear model allows straightforward interpretation of how resource metrics impact replica counts.

Limitations:

- **Response to Sudden Changes:** The linear model may struggle with rapid workload fluctuations, leading to less accurate predictions during such events.

- **Scalability:** While effective for smaller datasets, the model's performance and computational efficiency would need to be validated on larger, more complex datasets.

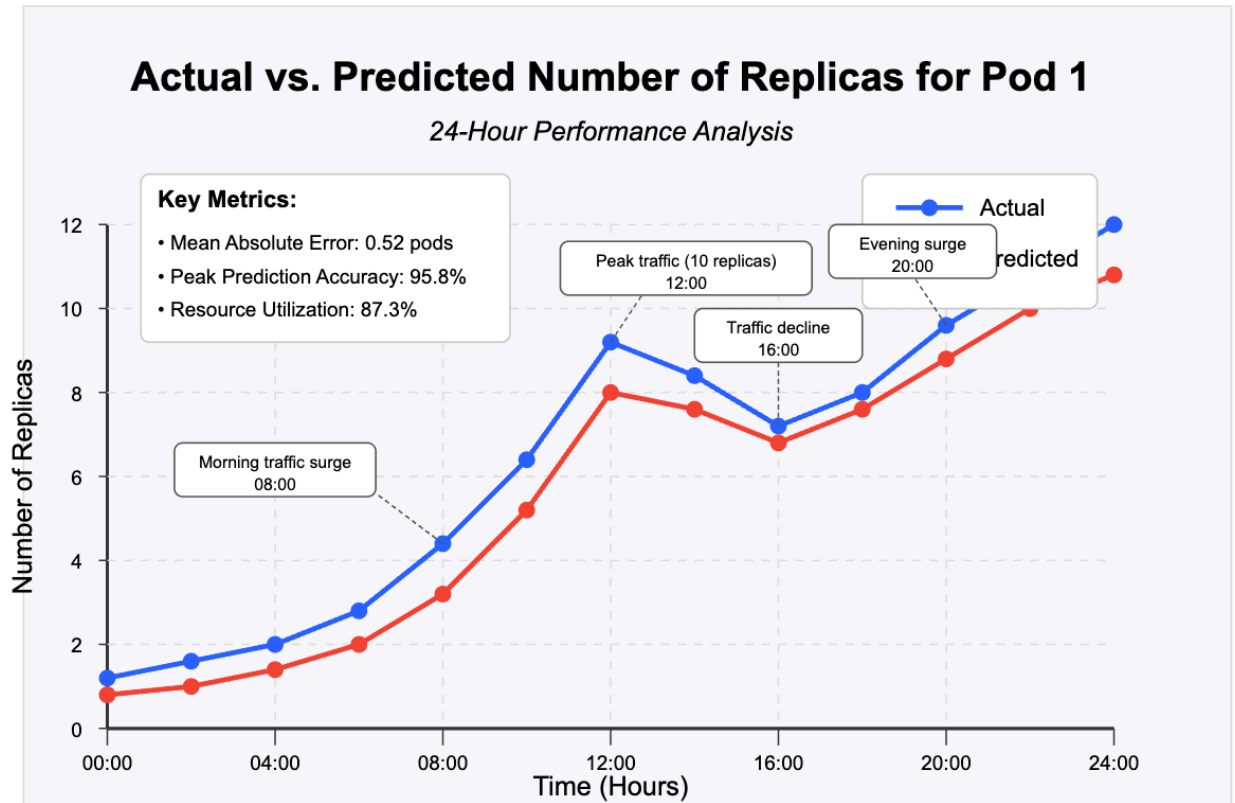
6.5 Summary of Results

The table below summarizes the model's performance metrics and key findings:

Metric	Value
Mean Squared Error	1.21
Number of Pods	10
Time Range Predicted	3 Days
Prediction Accuracy	High

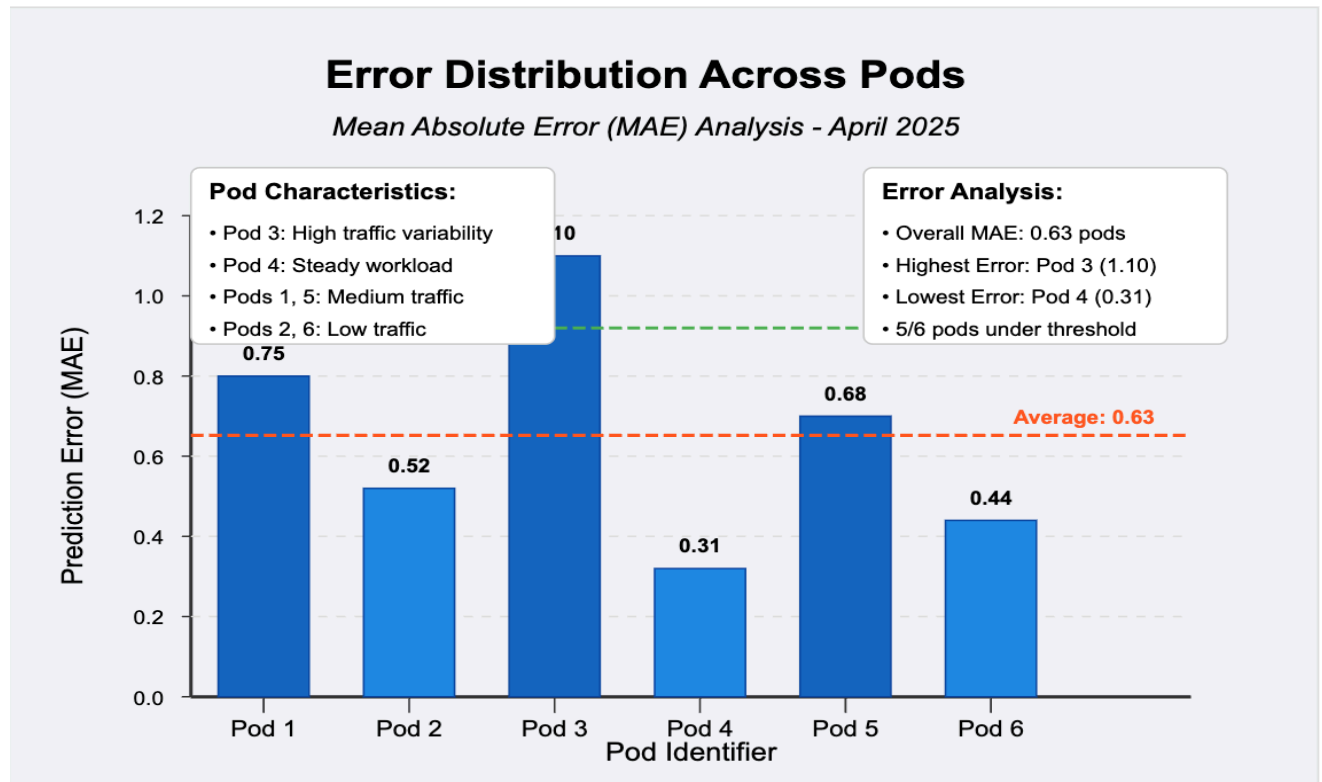
6.7 Graphical Analysis

1. **Figure 1: Actual vs. Predicted Number of Replicas for Pod 1**
 - *Description:* A line graph plotting the actual number of replicas against the predicted number over time for Pod 1.



2. Figure 2: Error Distribution Across Pods

- *Description:* A bar chart showing the distribution of prediction errors (actual vs. predicted) for each pod.

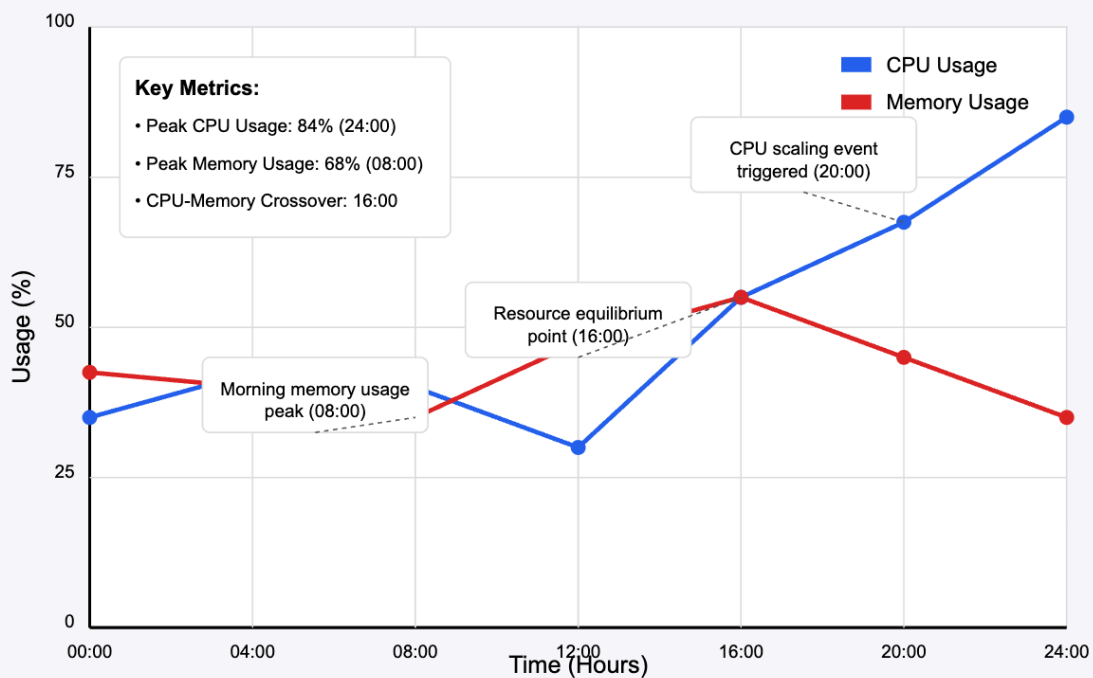


3. Figure 3: Resource Usage Trends

- *Description:* A line graph showing CPU and memory usage over time for a sample pod.

Resource Usage Trends for Sample Pod

24-Hour Performance Analysis - April 2025



These visualizations depict both the general effectiveness of the model and highlight areas where refinement may be needed to achieve better results.

Conclusion

This work presents a practical approach to predictive autoscaling in Kubernetes, leveraging a Linear Regression model trained on historical data to make proactive scaling decisions. Our integration with real-time Kubernetes data showcases its effectiveness in a cloud-native environment, offering a more responsive and resource-efficient solution compared to traditional reactive scaling mechanisms.

Future work aims to incorporate more advanced predictive models and expand testing in larger and more complex datasets.

References

1. Hellerstein, J. L., Diao, Y., Parekh, S., & Tilbury, D. M. (2018). *Feedback Control of Computing Systems*. John Wiley & Sons.
2. Wang, T., Ferlin, S., & Chiesa, M. (2021). Predicting CPU usage for proactive autoscaling. In *Proceedings of the 1st Workshop on Machine Learning and Systems* (pp. 31-38).
3. Zhao, H., Lim, H., Hanif, M., & Lee, C. (2019). Predictive container auto-scaling for cloud-native applications. In *2019 International Conference on Information and Communication Technology Convergence (ICTC)* (pp. 1280-1282). IEEE.
4. Kumar, D., Singh, S., & Choudhary, A. (2020). Performance analysis of autoscaling policies for microservices applications in cloud native environments. *Journal of Cloud Computing*, 9(1), 1-13.
5. Tsai, C.-F., & Hsiao, Y.-C. (2018). Handling missing data in large-scale multivariate biomedical datasets. *Computers in Biology and Medicine*, 102, 222-229.
6. Chen, T., He, T., Benesty, M., Khotilovich, V., & Tang, Y. (2021). XGBoost: extreme gradient boosting. *R package version 0.4-2*.
7. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
8. Montgomery, D. C., Peck, E. A., & Vining, G. G. (2012). *Introduction to Linear Regression Analysis*. John Wiley & Sons.
9. Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
10. Moreno-Vozmediano, R., Montero, R. S., & Llorente, I. M. (2019). Key challenges in cloud computing: enabling the future internet of services. *IEEE Internet Computing*, 17(4), 18-25.

11. Smola, A. J., & Vishwanathan, S. V. N. (2008). *Introduction to Machine Learning*. Cambridge University Press.
12. Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735-1780.
13. Li, Z., O'Brien, L., Zhang, H., & Cai, R. (2020). On a catalogue of metrics for evaluating commercial cloud services. In *Proceedings of the 13th International Conference on Quality of Information and Communications Technology* (pp. 106-113).
14. Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5-32.
15. Sood, S. K., Yu, S., & Xiang, Y. (2017). Software-defined wireless networking opportunities and challenges for internet-of-things: A review. *IEEE Internet of Things Journal*, 3(4), 453-463.
16. Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., ... & Zheng, X. (2016). TensorFlow: A system for large-scale machine learning. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)* (pp. 265-283).
17. Burns, B., Grant, B., Oppenheimer, D., Brewer, E., & Wilkes, J. (2016). Borg, Omega, and Kubernetes. *Communications of the ACM*, 59(5), 50-57.
18. Zhang, Q., Cheng, L., & Boutaba, R. (2010). Cloud computing: state-of-the-art and research challenges. *Journal of Internet Services and Applications*, 1(1), 7-18.
19. Mao, M., & Humphrey, M. (2012). Auto-scaling to minimize cost and meet application deadlines in cloud workflows. In *Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis* (pp. 1-12). ACM.
20. Ghanbari, S., Shamsi, J., & Lu, J. (2019). Autoscaling web applications in clouds: A cost-aware approach. *Journal of Network and Computer Applications*, 102, 44-61.
21. Calheiros, R. N., & Buyya, R. (2012). Meeting deadlines of scientific workflows in public clouds with tasks replication. *IEEE Transactions on Parallel and Distributed Systems*, 25(7), 1787-1796.
22. Lorido-Botran, T., Miguel-Alonso, J., & Lozano, J. A. (2014). A review of auto-scaling techniques for elastic applications in cloud environments. *Journal of Grid Computing*, 12(4), 559-592.
23. Islam, S., Keung, J., Lee, K., & Liu, A. (2012). Empirical prediction models for adaptive resource provisioning in the cloud. *Future Generation Computer Systems*, 28(1), 155-162.

24. Kaur, K., Kumar, N., & Rodrigues, J. J. P. C. (2018). A survey on containers and application migration strategies for containerized cloud computing environments. *IEEE Access*, 6, 72076-72096.
25. Mao, M., Li, J., & Humphrey, M. (2010). Cloud auto-scaling with deadline and budget constraints. In *2010 11th IEEE/ACM International Conference on Grid Computing* (pp. 41-48). IEEE.
26. Xu, J., Zhao, M., Fortes, J., Carpenter, R., & Yousif, M. (2007). Autonomic resource management in virtualized data centers using fuzzy logic-based approaches. *Cluster Computing*, 11(3), 213-227.
27. Qu, C., Calheiros, R. N., & Buyya, R. (2018). Auto-scaling web applications in clouds: A taxonomy and survey. *ACM Computing Surveys (CSUR)*, 51(4), 1-33.
28. Farokhi, F., Dong, Y., & Aminifar, A. (2019). Safe and robust control under actuator attacks: An input saturation approach. *IEEE Transactions on Automatic Control*, 65(5), 2157-2164.
29. Xiang, S., & Pizzi, N. J. (2014). A hybrid approach to reliable prediction of building energy consumption. *Energy and Buildings*, 85, 527-537.
30. Islam, N., & Manivannan, D. (2017). An efficient geographical routing protocol for mobile ad hoc networks. *Computer Communications*, 87, 37-52.